最初的博弈

game | 同中心胜负稳定性与动态线段树 | C++17

1. 用比较符号表示单调性

定义 s_i=sign(a_(i+1)-a_i)，取值为 -1、0、1。区间 [l,r] 非严格单调，当且仅当 s_l,...,s_(r-1) 不同时包含 1 和 -1。

删除一个端点相当于删去最左或最右的一个比较符号。用 W(l,r) 表示当前轮到的人是否必胜；单调区间是必败状态，因为题目规定初始单调时由后手获胜。

```
W(l,r) = 0 if monotone
W(l,r) = not W(l+1,r) or not W(l,r-1) otherwise
```

2. 跨两层的递推

若 [l,r] 的两个长度少一的子区间都不单调，对它们再展开一次递推，令 A=W(l+2,r)、B=W(l+1,r-1)、C=W(l,r-2)，可化简为：

```
W(l,r) = B and (A or C)
```

推导：两个孩子的胜负分别为 not A or not B、not B or not C；当前状态取二者取反后的或，即 (A and B) or (B and C)。

如果至少一个孩子单调，则当前状态直接必胜，不使用这一化简式。下面的证明始终区分这两种情况。

3. 同中心扩张

把 [l,r] 扩张为 [l-1,r+1]，中心不变、长度增加 2。只要 [l,r] 已经不单调，扩张后两个长度少一的孩子都包含 [l,r]，也不单调，因此可以使用上式。

只需检查中心附近两层

4. 必败沿同中心扩张保持

若一个不单调区间 W(l,r)=0，那么其同中心扩张的递推中 B=0，故扩张后仍为必败。反复扩张依然成立。

5. 一对相邻必胜区间会一直保持

考虑相同长度、向右平移一格的两个区间 I=[l,r]、J=[l+1,r+1]，两者都必胜。把它们各自向左右扩张一格，得到 I+、J+。

对 I+，若某个孩子单调则它直接必胜；否则两层递推中，中心 B 是 I，侧边 A、C 中有一个是 J。因为 I、J 都必胜，I+ 必胜。对 J+ 同理。这一对相邻区间可以继续成对扩张，始终必胜。

6. 找到最内层不单调区间 I

从询问区间 [L,R] 同时删除两端，直到再删一对端点就会单调。记最后保留的不单调区间为 I=[l,r]，称为中心核心。

若核心必败，根据第 4 节，原询问必败。若核心必胜且它就是原询问，则原询问必胜。剩下只需要判断核心向外扩一层的 I+。

I+ 的两个孩子都包含非单调的 I，所以两层递推适用。它必胜当且仅当核心同长度的两个侧邻区间 [l-1,r-1]、[l+1,r+1] 至少一个必胜。

若两侧都必败，I+ 必败，所有更外层也必败。若某侧必胜，它与核心组成一对相邻必胜区间；由第 5 节，这一对一直向外保持必胜，所以原询问必胜。

因此每次询问只需求核心，以及它的两个侧邻区间的胜负，完全不需要建立平方级区间 DP。

怎样快速找到核心

7. 每条比较边能承受多少对称删除

对于询问 [L,R] 内的一条比较边 i，删除两端各 d 个数后，它仍存在当且仅当 L+d<=i<=R-d-1。所以它最多能承受的删除次数是：

```
h(i) = min(i-L, R-i-1)
```

分别在正比较边和负比较边中求最大的 h，记为 h_+、h_-。若缺少任意一种，原区间单调，回答 Bob。否则最多还能同时保留两种符号的删除次数为 d=min(h_+,h_-)。

```
l = L+d, r = R-d
```

h(i) 是以比较边区间中点为峰值的三角形函数。每种符号只需检查中点左侧最后一条边，以及中点右侧第一条边。用线段树查询这四个位置即可，不需要二分或逐步收缩。

8. 核心的胜负只看两条边

核心内部 [l+1,r-1] 单调。若两条边 s_l、s_(r-1) 相同且非零，因为核心不单调，内部必有相反符号。删掉任意一个端点，仍同时含有两种符号；而两个孩子再删除靠外端点就单调，因此孩子都必胜，核心必败。

否则，至少一个孩子单调，核心可以一步获胜。理由是内部最多有一种非零符号，核心的非单调性只能由外侧边补足相反符号。若删哪边都仍非单调，两侧边就必须同时等于这个相反的非零符号，恰好回到上一种情况。

```
core loses iff s[l] == s[r-1] and s[l] != 0
```

侧邻区间的常数次判定

9. 左侧邻区间的特殊形状

左侧邻区间是 J=[l-1,r-1]，删掉左端两个数后得到核心内部 [l+1,r-1]，它单调。一般地，考虑一个区间 [x,y]，且 [x+2,y] 单调。

记 R(x) 为从 x 开始向右的最长单调区间右端点；令 R0=R(x)，R1=R(x+1)。由于单调性对取子区间保持，R0<=R1。

若 y<=R0，则区间本身单调，必败。若 R0<y<=R1，则删除左端即可单调，必胜。若 y>R1，删除左端后的区间非单调，但再删除左端就单调，所以这个孩子必胜。当前胜负只由右侧孩子取反得到。

```
W(x,y) = not W(x,y-1) when y>R1
```

在 y=R1 处，若 R0<R1，状态必胜；若 R0=R1，状态单调而必败。于是第三种情况由奇偶性直接给出：

```
W(x,y) = ((y-R1) mod 2 == [R0==R1])
when y>R1 and [x+2,y] is monotone
```

10. 从核心的首末符号位置得到 R0、R1

查询核心比较边中第一条正边 fp 和第一条负边 fm。两种符号首次同时出现时，较晚出现的那条边就是最长单调前缀的右端点。因此对左邻区间，R1=max(fp,fm)。再把边 l-1 加入两种符号的首次位置候选，即得 R0。

只需将上述公式应用到 x=l-1、y=r-1。查询范围以外的后续边不影响判定：我们只需要最长单调前缀端点是否已经小于 y；核心本身保证 fp、fm 都存在。

对称公式与完整算法

11. 右侧邻区间

右侧邻区间是 [l+1,r+1]，删掉最右两个数后单调。对称地，用 L(y) 表示以 y 结尾的最长单调后缀左端点。令 L0=L(r+1)、L1=L(r)，有 L0>=L1。

设 X=l+1。若 L0<=X，区间单调，必败；若 L1<=X<L0，删除右端立即获胜；否则其胜负是：

```
W(X,r+1) = ((L1-X) mod 2 == [L0==L1])
when X<L1
```

查询核心中最后一条正边 lp 和最后一条负边 lm，可得 L1=min(lp,lm)+1。再把边 r 加入末次位置候选，取较早者加一，得到 L0。

12. 线段树维护内容

对比较边序列建线段树。每个节点存正边首次位置、负边首次位置、正边末次位置、负边末次位置。合并时首次位置取最小值，末次位置取最大值；不存在时使用正负哨兵。

询问先查询中点左右的边区间，取各符号最靠中间的位置，求出 d 及核心。若核心必败或 d=0，即可返回。否则再查询核心的四个首末位置，代入两个侧邻公式，取它们胜负的或。

修改 a_x 只会影响比较边 x-1 和 x，重新计算这两条边并单点更新即可。

13. 复杂度

```
build: O(n)
update: O(log n)
query: O(log n)
total: O(n + q log n), space: O(n)
```

一次询问最多做三次线段树范围查询。n,q<=10^6，使用迭代线段树及四个整数的位置摘要，内存远低于 256MB。所有比较直接比较数值，不做有溢出风险的差值运算。

实现边界与验证

14. 边界检查

长度 1、2 的序列总单调，直接回答 Bob。全相等、非严格递增、非严格递减的区间均缺少某一类比较符号，同样回答 Bob。零比较边保留原位置，不能直接删除，否则会改变轮流操作的奇偶性。

只有 d>0 时才访问核心外侧边 l-1、r。此时这两条边均位于原询问中，不会越界。不存在的比较符号必须在计算 h(i) 前检查范围，不能让哨兵被当成合法边。

代码默认读取 game.in 并写入 game.out；编译时加 -DLOCAL 可使用标准输入输出。更新和查询顺序完全在线处理，不依赖特殊性质 B。

15. 样例

题面样例的四次询问输出依次为 Bob、Alice、Bob、Alice。初始 [1,4] 单调，所以由 Bob 获胜。其余询问由同一胜负递推统一处理；修改值后只需更新邻接的比较边。

16. 独立验证记录

枚举所有长度不超过 10 的比较符号序列（对应原数组长度不超过 11），对每个序列枚举全部区间，用直接区间 DP 的定义递推核对线段树算法。每种比较符号序列都可用合法正整数数组实现。

另做 2000 组随机数组，每组 50 次修改，每次修改后重新建立小规模区间 DP，并核对 10 次随机区间询问。两部分累计 6380840 次区间检查全部通过；随机种子 20261011。

题面样例在标准输入输出与文件 IO 两种模式下均通过。另完成 n=q=10^6 的在线混合操作运行检查。正确性依据是两层递推、相邻必胜区间稳定性及侧邻奇偶公式，测试通过不等同于官方 AC。